Task-4:-Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the technical and functional requirements for deploying three frontend applications (Air-Quality, Bookshop, and Notesapp) across multiple AWS services. This document is intended for DevOps engineers, cloud architects, and stakeholders involved in standardizing the organization's frontend deployment pipeline.

1.2 Scope

- **In Scope:** Dockerization of three frontend apps, Nginx configuration for EC2, S3 static website hosting setup, ECS Fargate/EC2 task definitions, ECR image management, and a comparative analysis of the three services.
- **Out Scope:** Backend API development, database migration, and domain registration/DNS management.

1.3 Definitions, Acronyms, and Abbreviations

- **EC2:** Elastic Compute Cloud
- **ECS:** Elastic Container Service
- **ECR:** Elastic Container Registry
- **S3:** Simple Storage Service
- **ALB:** Application Load Balancer
- **SPA:** Single Page Application

1.4 References

- Project Repository: `pushkar-iamops/frontend-apps`
- AWS Documentation: Amazon S3 Hosting, Amazon ECS Guide, Nginx Documentation.

1.5 Overview

This document outlines the deployment strategy for three unique frontend projects, visualizes the user interaction via use cases, establishes performance and security benchmarks, and concludes with a risk assessment for the proposed infrastructure.

---

2. Project Overview

2.1 Project Objectives

- Deploy three frontend applications on EC2, ECS, and S3.
- Automate containerization using Docker and ECR.
- Compare hosting methods to determine a single organizational standard.

2.2 Business Goals

- **Scalability:** Identify the service that best handles traffic spikes.
- **Cost Optimization:** Minimize monthly AWS spend by choosing the most efficient hosting model.
- **Operational Simplicity:** Reduce the time developers spend on infrastructure management.

2.3 Background

Currently, frontend deployments are fragmented. By testing the **Air-Quality**, **Bookshop**, and **Notesapp** projects on three different AWS compute tiers, the team can gather empirical data on performance and ease of use.

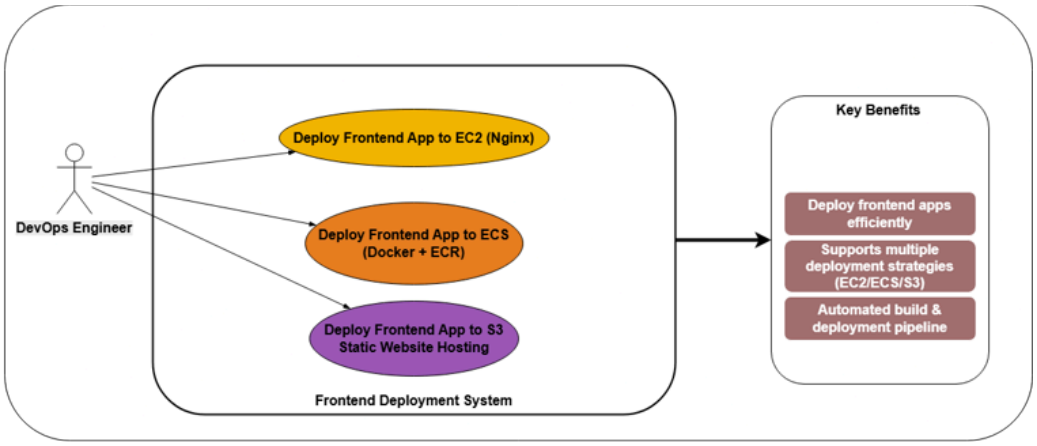
2.4 Constraints

- **Time:** All deployments and the final report must be completed within a 5-hour window.
- **Technology:** Must utilize Nginx for EC2 and Docker for ECS.

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Static Website Deployment (S3)

- **Description:** Building the app for production and hosting it as a static site.

- **Actors:** DevOps Engineer.
- **Preconditions:** Source code is available; AWS CLI is configured.
- **Postconditions:** App is accessible via an S3 website endpoint.
- **Flow of Events:**
  a. Run `npm run build` to generate static artifacts.
  b. Create an S3 bucket with public access or CloudFront distribution.
  c. Upload files using `aws s3 sync`.
- **Alternative Flows:** If the build fails, fix dependencies and retry.

3.2.2 Use Case 2: Containerized Deployment (ECS)

- **Description:** Creating a Docker image and running it as a managed service.
- **Actors:** DevOps Engineer, AWS ECS.
- **Preconditions:** Dockerfile exists; ECR repository created.
- **Postconditions:** App is running in a container managed by ECS.
- **Flow of Events:**
  a. Build Docker image: `docker build -t app-name .`
  b. Push image to Amazon ECR.
  c. Create Task Definition and ECS Service.
- **Alternative Flows:** If the container fails health checks, roll back to the previous image version.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- **Loading Speed:** S3-hosted apps should have a Time to First Byte (TTFB) under 200ms when used with a CDN.
- **Scaling:** ECS should be configured to scale based on CPU utilization.

4.2 Security Requirements

- **Data Protection:** Use SSL/TLS (HTTPS) for all public-facing endpoints.
- **Access Control:** S3 buckets should use Bucket Policies to restrict access to specific referrers or CloudFront.

4.3 Usability Requirements

- **Accessibility:** Each app must be reachable via a public URL without requiring an internal VPN.

4.4 Reliability Requirements

- **Uptime:** Target 99.9% availability for the production environment.
- **Recovery:** ECS tasks should automatically restart upon failure.

---

5. Assumptions and Dependencies

5.1 Assumptions

- All three applications are primarily client-side (React/Vue/Angular/HTML).
- Public internet access is available for downloading dependencies.

5.2 Dependencies

- **Docker:** Necessary for the ECS deployment path.
- **Nginx:** Necessary for the EC2 deployment path.

---

6. Risk Analysis

6.1 Risk Identification

- **High Costs:** Running three separate EC2 instances for small frontend apps might be expensive.
- **Image Bloat:** Large Docker images leading to slow deployment times in ECS.

6.2 Risk Mitigation

- Use **Multi-stage Docker builds** to keep image sizes small.
- Utilize **S3 Static Hosting** for purely static apps to minimize costs.

---

7. Glossary

- **Bucket:** A container for objects stored in Amazon S3.
- **Fargate:** A serverless compute engine for containers.
- **Reverse Proxy:** A server (Nginx) that sits in front of web servers and forwards client requests.

---

8. Appendices

- Dockerfiles for Air-Quality, Bookshop, and Notesapp.
- Nginx configuration snippets for EC2.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____